

Отчёт по лабораторной работе №4

**Математические основы защиты информации и информационной
безопасности**

Ли Хан

Содержание

1	Цель работы	5
2	Анализ логики алгоритмов	6
2.1	Алгоритм Евклида	6
2.1.1	Алгоритм Евклида компилятора	7
2.1.2	результат	7
2.2	Бинарный алгоритм Евклида	7
2.2.1	Бинарный алгоритм Евклида компилятора	8
2.2.2	результат	9
2.3	Расширенный алгоритм Евклида	9
2.3.1	Расширенный алгоритм Евклида компилятора	10
2.3.2	результат	10
2.4	Расширенный бинарный алгоритм Евклида	11
2.4.1	Расширенный бинарный алгоритм Евклида компилятора	12
2.4.2	результат	13
3	Вывод	14

Список иллюстраций

2.1	Алгоритм Евклида компилятора	7
2.2	Алгоритм Евклида результат	7
2.3	Бинарный алгоритм Евклида компилятора	8
2.4	Бинарный алгоритм Евклида результат	9
2.5	Расширенный алгоритм Евклида компилятора	10
2.6	Расширенный алгоритм Евклида результат	10
2.7	Расширенный бинарный алгоритм Евклида компилятора	12
2.8	Расширенный бинарный алгоритм Евклида результат	13

Список таблиц

1 Цель работы

Изучить принципы различных алгоритмов поиска наибольшего общего делителя (НОД), включая алгоритм Евклида и его модификации. На практике реализовать и сравнить логику алгоритмов при работе с целыми числами, а также освоить нахождение линейной комбинации (соотношение Безу) с помощью расширенных алгоритмов.

2 Анализ логики алгоритмов

2.1 Алгоритм Евклида

Классический метод нахождения НОД. Основан на последовательном делении с остатком до тех пор, пока остаток не станет равен нулю.

Шаги реализации:

- Инициализация: Присвоить $r_0 = a, r_1 = b$.
- Деление: Найти остаток r_{i+1} от деления r_{i-1} на r_i .
- Проверка: Если $r_{i+1} = 0$, алгоритм завершается
- Результат: НОД равен r_i . В противном случае повторить шаг 2.

2.1.1 Алгоритм Евклида компилятора

```
def euclidean_gcd(a, b):  
    # 中文: 初始化余数 r0 为 a, r1 为 b  
    # RU: Инициализация r0 = a, r1  
    r0, r1 = a, b  
  
    # 中文: 循环执行取余操作, 直到余数 r1 变为 0  
    # RU: Цикл продолжается, по  
    while r1 != 0:  
        # 中文: 核心逻辑: r_{i+1} = r_{i-1} mod r_i  
        # RU: Основная логика:  
        r0, r1 = r1, r0 % r1  
  
    # 中文: 返回最后的非零余数作为最大公约数  
    # RU: Возвращает последний  
    return r0
```

Рис. 2.1: Алгоритм Евклида компилятора

2.1.2 результат

```
# 样例运行 / Пример запуска  
# 输入 (12345, 24690) -> 输出: 12345  
print(f"НОД (12345, 24690) = {euclidean_gcd(12345, 24690)}")  
  
НОД (12345, 24690) = 12345
```

Рис. 2.2: Алгоритм Евклида результат

2.2 Бинарный алгоритм Евклида

Более быстрый алгоритм для компьютерной реализации. Использует свойства четности чисел и заменяет деление на сдвиги и вычитание.

Шаги реализации:

- Общий множитель: Вынести общую степень двойки в переменную g .

- Устранение четности: Делить u и v на 2, пока они не станут нечетными
- Итерация: Если $u \geq v$, положить $u = u - v$, иначе $v = v - u$.
- Финал: Когда $u = 0$, итоговый $\text{gcd} = g \cdot v$.

2.2.1 Бинарный алгоритм Евклида компилятора

```
def binary_gcd(a, b):
    # 中文: g 用于记录提取出的公因子 2
    # RU: g  используется для хранения
    g = 1

    # 中文: 当 a 和 b 都是偶数时, 不断提取公因子 2
    # RU: Пока a и b четные, выносим
    while a % 2 == 0 and b % 2 == 0:
        a //= 2
        b //= 2
        g *= 2

    u, v = a, b
    # 中文: 主循环, 通过移位和减法代替取模运算
    # RU: Основной цикл: сдвиги и вы
    while u != 0:
        while u % 2 == 0: u //= 2
        while v % 2 == 0: v //= 2
        if u >= v:
            u = u - v
        else:
            v = v - u

    # 中文: 结果为提取出的 2 的幂次与剩余值的乘积
    # RU: Результат — произведение ;
    return g * v
```

Рис. 2.3: Бинарный алгоритм Евклида компилятора

2.2.2 результат

```
# 样例运行 / Пример запуска
# 输入 (48, 18) -> 输出: 6
print(f"Бинарный НОД (48, 18) = {binary_gcd(48, 18)}")
```

Бинарный НОД (48, 18) = 6

Рис. 2.4: Бинарный алгоритм Евклида результат

2.3 Расширенный алгоритм Евклида

Позволяет найти не только НОД d , но и целые числа x, y , такие что $ax + by = d$.

Шаги реализации:

- Инициализация: Задать начальные значения остатков r и коэффициентов x, y .
- Деление с остатком: Найти частное q_i и остаток r_{i+1} .
- Пересчет: Обновить коэффициенты x_{i+1} и y_{i+1} через полученное частное q_i .
- Завершение: При $r_{i+1} = 0$ выдать результат d, x, y .

2.3.1 Расширенный алгоритм Евклида компилятора

```
def extended_euclidean_gcd(a, b):
    # 中文: 初始化系数 x 和 y, 满足 ax + by = r
    # RU: Инициализация коэффициентов
    r0, r1 = a, b
    x0, x1 = 1, 0
    y0, y1 = 0, 1

    while r1 != 0:
        # 中文: 计算商 q 用于后续系数更新
        # RU: Находим частное q д:
        q = r0 // r1
        r0, r1 = r1, r0 - q * r1
        # 中文: 更新线性组合的系数
        # RU: Обновление коэффициентов
        x0, x1 = x1, x0 - q * x1
        y0, y1 = y1, y0 - q * y1

    # 中文: 返回最大公约数 d 以及系数 x, y
    # RU: Возвращает НОД d и коэффициенты
    return r0, x0, y0
```

Рис. 2.5: Расширенный алгоритм Евклида компилятора

2.3.2 результат

```
# 样例运行 / Пример запуска
# 输入 (91, 105) -> 输出: (7, 7, -6) -> 即 91*7 + 105*(-6) = 7
d, x, y = extended_euclidean_gcd(91, 105)
print(f"Расширенный НОД: d={d}, x={x}, y={y}")
```

Расширенный НОД: d=7, x=7, y=-6

Рис. 2.6: Расширенный алгоритм Евклида результат

2.4 Расширенный бинарный алгоритм Евклида

Модификация бинарного алгоритма для поиска коэффициентов x, y без использования операции деления.

Шаги реализации:

- Подготовка: Извлечь общую степень двойки g .
- Сдвиги: Пока u или v четные, выполнять деление на 2.
- Корректировка: Если коэффициенты нечетные, прибавить к ним исходные числа a или b перед делением на 2.
- Вычитание: Обновлять значения u, v и коэффициенты до обнуления u .

2.4.1 Расширенный бинарный алгоритм Евклида компилятора

```
def extended_binary_gcd(a, b):
    g = 1
    while a % 2 == 0 and b % 2 == 0:
        a //= 2; b //= 2; g *= 2

    u, v = a, b
    # 中文: A, B, C, D 对应线性组合的辅助变量
    # RU: A, B, C, D — вспомогательные переменные
    A, B, C, D = 1, 0, 0, 1

    while u != 0:
        while u % 2 == 0:
            u //= 2
            # 中文: 若系数为偶数则除以 2, 否则按公式调整
            # RU: Если коэффициенты четные
            if A % 2 == 0 and B % 2 == 0:
                A //= 2; B //= 2
            else:
                A = (A + b) // 2; B = (B - a) // 2
        # ... 对 v 执行类似逻辑 ...
        while v % 2 == 0:
            v //= 2
            if C % 2 == 0 and D % 2 == 0:
                C //= 2; D //= 2
            else:
                C = (C + b) // 2; D = (D - a) // 2

        if u >= v:
            u -= v; A -= C; B -= D
        else:
            v -= u; C -= A; D -= B

    return g * v, C, D
```

Рис. 2.7: Расширенный бинарный алгоритм Евклида компилятора

2.4.2 результат

```
# 样例运行 / Пример запуска
# 输入 (154, 105) -> 输出: (7, 1, -1) -> 即  $154*1 + 105*(-1) = 49$  (示
d, x, y = extended_binary_gcd(154, 105)
print(f"Бинарный расширенный НОД: d={d}, x={x}, y={y}")
```

Бинарный расширенный НОД: d=7, x=13, y=-19

Рис. 2.8: Расширенный бинарный алгоритм Евклида результат

3 Вывод

Экспериментально подтверждено, что классический алгоритм Евклида является наиболее простым в реализации, в то время как бинарный алгоритм эффективнее на низком уровне за счет исключения деления. Расширенные алгоритмы позволяют не только найти НОД, но и представить его в виде линейной комбинации исходных чисел, что критически важно для решения диофантовых уравнений.